From Sparse Body Inputs to Offline Garment Size Guidance: A Source-Level Study of an Android Size Guide

Ikbal Fadilah
Causyne
ikbal@causyne.com
ORCID: 0009-0008-4853-135X

26 October 2026

Abstract

Choosing clothing by a nominal label is unreliable when charts differ across retailers, garment types, and markets. This paper examines the source of an Android size guide designed to operate with few body inputs and an optional photographed size chart. The active recommendation route accepts height, weight, gender and region, allows direct circumferences when available, estimates missing dimensions with deterministic formulae, obtains either a built-in general chart or a user-provided chart, validates the inputs, and ranks garment sizes by dimension-level fit scores. A separate image route uses bundled Google ML Kit text recognition and reconstructs chart cells from text geometry before the same recommendation stage. The paper distinguishes implemented behavior from intended behavior: current body estimation deliberately skips a trained predictor, its regional multipliers conflict with a comment elsewhere describing globally invariant body measurements, and the chart parser uses heuristics rather than a measured guarantee of OCR accuracy. No fit-accuracy or latency benchmark is claimed. We give an auditable map from UI actions to source functions, derive the scoring calculation from code, identify failure modes, and specify an evaluation protocol for future empirical work. A selected, unchanged source snapshot accompanies the manuscript; it is a research extract from the original Android application, not an independently buildable application.

Keywords: clothing size recommendation; on-device OCR; Android; fit scoring; source-level study; offline computing

1 Introduction

A person can know that one shirt labelled L fits and still have little basis for choosing an L from another retailer. Nominal sizes compress several measurements into a single symbol; the underlying chest, waist, hip, shoulder, ease and cut may differ. The problem also changes with garment type: a forgiving hoodie and structured blazer should not interpret the same gap between body and garment measurements identically. Regional labels add a second ambiguity. The motivating examples for this project came from buying clothing across brands such as Zara and Uniqlo and encountering differences across national markets. These examples motivate the problem; they do not establish a universal conversion such as "an L in one country equals an XL in another."

The project began as a practical attempt to reduce the amount of information a shopper must provide. Height and weight are often known, whereas accurate chest, waist and hip

measurements may not be. A user may also have a photographed chart instead of machine-readable size data. The implementation therefore contains two paths: sparse-input general guidance and chart-specific guidance. Both culminate in a numerical ranking over size entries. An optional image path extracts a table from a photographed chart on the device. The system is useful to study because it exposes its intermediate assumptions, including the distinction between a user's measured dimension, an estimated dimension and an OCR-derived garment measurement.

Existing research approaches the problem differently. Guigourès and colleagues modeled purchase and return outcomes with a hierarchical Bayesian recommender [1]. That setting has behavioral data at scale; the Android implementation studied here is primarily a local, rule-based and numerical pipeline. ISO 8559-1 defines anthropometric measurements for clothing work [2]. It does not certify the application's formula coefficients or retailer tables. Google ML Kit provides Android text-recognition primitives [3]; it does not itself solve table topology or product-specific size interpretation. This paper describes the engineering of those links and the limits of what source inspection can establish.

Our contribution is deliberately bounded. We trace the actual active call graph in the original Android source, not a newly implemented surrogate. We state equations only where supported by code. We separate present behavior from unused components and future plans, and offer a reproducible audit path in the companion repository. The paper does not report an unperformed user study, a validated size prediction rate, a garment-return reduction or a comparison with production recommenders.

2 Evidence and scope

The object of study is the Size Guide code copied without edits from the author's Android application on 26 October 2026. The snapshot includes the size presentation layer, its View-Model, domain models, chart reconstruction, estimation, scoring, explanation and selected tests. Its SHA-256 manifest records each copied file. It deliberately omits unrelated application components, server credentials, deployment settings, private user data and third-party raw datasets. These omissions limit standalone compilation: the extract retains imports from the original app, including dependency injection and an optional network service type. The complete original app is the environment in which the feature was developed.

Source inspection can establish which functions call which other functions and what deterministic code computes for a specified input. It cannot, by itself, show that a shopper receives a correct size. A comment that says "validated" is a development assertion, not experimental validation. Tests in the snapshot are useful evidence of intended cases but do not substitute for a representative retailer-by-body dataset. We use the terms *implemented*, *reachable* and *measured* separately throughout.

Table 1 summarizes the modules most central to the active route. File paths in the repository retain the original package directories so that a reader can compare the manuscript with the copied bytes. The legacy package name is part of the historical source and has not been renamed in the research snapshot.

Table 1: Source map for the present study. Names are exact Kotlin filenames.

Component	Observed role
`SizeScreen`, `SizeViewModel`	Gather inputs, handle image selection, select a chart, invoke estimation and recommendation.
`BodyEstimator`	Validate sparse inputs; use provided measurements or deterministic estimates.
`GeneralSizeChartProvider`	Supply an embedded chart when no product chart is present.
`OcrSizeIntelligenceEngine`	ML Kit text extraction; token, cell and measurement reconstruction.
`InputValidator`, `PreScoringValidator`	Reject malformed inputs or structurally unsuitable charts before scoring.
`FitScoringEngineVNext`, `SizeRecommendationEngine`	Calculate fit scores, rank sizes, choose alternatives and report warnings.

3 End-to-end application route

The Size Guide UI retains input state in `SizeViewModel`. For the sparse-input path, the ViewModel validates a `UserBodyInput`; if no scanned or manually entered chart is present, it requests a general chart from `GeneralSizeChartProvider` using region, garment type and gender. The default garment type is T-shirt when none has been selected. If a user chart is present, the ViewModel validates that chart before recommendation. Direct body measurements are separately checked and used if valid. The ViewModel calls the estimator, then passes the estimated body and chart to the recommendation engine. The current call supplies `FitPreference.REGULAR`.

This branch distinction matters. A recommendation against an embedded general chart is not a recommendation against an actual Zara, Nike or Uniqlo product. Product-specific conclusions require an actual product chart, whether entered manually or reconstructed from a photograph. A nominal size label appears in the result but is not the numeric quantity used for comparison. The code ranks `SizeChartEntry` objects on measurements and then returns their labels for the shopper to read.

The image route begins in `processOCRFromImage`, which calls `processOCRLocally`. That function constructs `OcrSizeIntelligenceEngine` without an API service or authentication arguments; it calls the engine under a 60-second timeout and stores returned size entries as the chart. The bitmap is recycled in a `finally` block. The engine contains an optional server-assisted parse branch, but that branch requires an API service, tier and device hash; those arguments are absent in this active local call. We call this observed route local without claiming that every Size Guide pathway in the entire application is offline.

An important difference between these routes is the location of uncertainty. The general chart path relies on embedded constants; its chief uncertainty is whether those constants represent a particular garment. The image path supplies product data, but the table structure and units must be inferred from imperfect OCR. The later scoring stage consumes either chart, so a visually plausible score can conceal an upstream chart error. The application surfaces some warnings; the existence of warnings does not establish that all errors are caught.

4 Sparse-input body estimation

4.1 Validation and overrides

The estimator requires finite height and weight, with code ranges 100–250 cm and 30–300 kg. It computes body mass index as BMI $= w/(h/100)^2$ and rejects values outside 10–60. Optional chest, waist, hip, shoulder and inseam measurements have separate finite-range checks. These bounds are software input guards, not a scientific declaration about all possible bodies. They exclude some users and need evaluation for accessibility and coverage.

For relevant dimensions, a supplied valid measurement replaces an estimated value; an inapplicable dimension is stored as zero with a corresponding `NOT_APPLICABLE` method flag. This provenance is important. A directly measured waist should not be represented as though it came from a height-weight formula, and a missing hip for an upper garment should not be interpreted as a measured zero. The estimator's result includes a per-dimension method map and a confidence structure. That confidence is an internal score, not an empirically calibrated probability of fit.

4.2 Formula route actually used

The active function explicitly skips predictions from trained ML data, with a source comment stating the data is corrupted. It applies formula-based estimates when optional measurements are absent. For example, before regional factors, the male chest estimate is $0.52h + 0.32w - 15$ and the female chest estimate is $0.48h + 0.35w - 8$, with h in cm and w in kg. The corresponding waist formulae are $0.45h + 0.28w - 12$ and $0.40h + 0.30w - 15$; hip formulae are $0.53h + 0.30w - 10$ and $0.54h + 0.33w - 5$. The functions multiply these bases by region-specific factors. The origin and external validity of the coefficients cannot be established from code inspection alone. These formulae should be treated as implementation parameters, not population regression results validated in this paper.

The code also estimates shoulder width and inseam and chooses relevant dimensions by garment type. Its cross-validation can lower the internal confidence score for suspicious combinations. These steps make the pipeline inspectable but do not eliminate ambiguity: two people with the same height and weight can have materially different body proportions. When a user provides a circumference, the override is therefore more informative than the default formula for that dimension, subject to measurement error.

There is a noteworthy internal inconsistency. `RegionalFitNormalizer` comments say body measurements are globally invariant and that regional context affects only chart interpretation and tolerance. The active `BodyEstimator` multiplies its formula outputs by region-dependent factors. The score stage also applies regional tolerance and a garment-size multiplier. Thus region can influence both the estimated body and the interpreted garment interval in current code. We report this as an observed inconsistency, not as an intentional or validated anthropometric correction. An ablation with fixed body measurements across region settings is needed before making any invariance claim.

5 From a size-chart image to numerical rows

ML Kit recognizes text elements and bounding boxes from an Android `Bitmap` [3]. The implementation iterates blocks, lines and elements, retaining nonblank element text and coordinates. It assigns a fixed 0.8 value to each raw token because the integrated API pathway does not

provide the confidence quantity the code expects per element. As a result, the initial value is a chosen default, not a measured OCR confidence. Any later multiplication of this value must be interpreted accordingly.

The engine then normalizes candidate tokens, groups their positions into rows and columns, classifies cells as size labels or measurement values, parses ranges and units, checks the number of sizes, injects region and gender context and combines internal confidence components. A chart image is therefore not simply converted to a string. The geometry of each token matters: two numbers on the same visual line can represent adjacent sizes, or chest and waist for one size, depending on headings and columns. Table reconstruction is a distinct computer-vision and document-analysis task; research on general table extraction, such as PubTables-1M, illustrates that cell topology cannot be reduced to raw text recognition alone [4].

Measurement parsing supports ranges such as 96–100, a single number with an explicit cm or inch mark, and a bare number. The latter is interpreted as cm at lower internal confidence. A single value is expanded to a range of roughly $x \pm 2$ in source. This expansion is an engineering heuristic, not a retailer-provided tolerance. A quoted inch mark is recognized and later fit scoring converts inch-valued ranges to cm with factor 2.54. A unit error therefore propagates directly into the fit score. The paper's earlier OCR-only companion study treats chart reconstruction in greater detail; the present work connects that front end to the body and recommendation path rather than replacing the prior manuscript.

The parser can return no sizes; the ViewModel then surfaces a warning or an error message. This is preferable to inventing a chart, but it does not prevent a wrongly structured nonempty chart. A row/column swap, duplicated size label, lost heading, wrong unit or OCR substitution can still yield plausible numerical entries. Validation of chart dimensions and size counts narrows this space but is not a proof of correctness. For scientific evaluation, OCR text accuracy, table structure accuracy and downstream sizing quality must be measured separately.

6 Measurement-based fit scoring

6.1 From chart interval to a dimension score

The active recommender calls `FitScoringEngineVNext` for each size entry. The scoring function obtains garment-specific weights, alters a measurement range according to garment-physics rules, converts inches to centimeters, applies a regional size multiplier and chooses an ideal point within the resulting interval. For a dimension d, the code then forms a distance between the user's estimated or measured value b_d and a target m_d. Its dimension score is Gaussian-like: $s_d = \exp[-k(\Delta_d/\tau_d)^2]$, where the code chooses tolerance τ_d from garment type, range width and region, and k is its steepness parameter. This formula describes the implemented ranking curve, not an observed probability distribution of garment fit.

The garment-type constants differ substantially. For example, the source assigns a narrower base tolerance to jeans than to a hoodie. Physics adjustments apply ease allowances before scoring. This is an explicit attempt to distinguish an intentionally loose garment from a close-fitting one. It also creates sensitivity to hand-chosen constants: a score may change even when the chart and body inputs remain fixed, solely because the garment category changes. Any empirical claim about such choices requires garments of multiple cuts and materials.

For each available dimension, the code multiplies the raw dimension score by a combined user-body and chart confidence. It computes a weighted average over applicable dimensions,

then applies a final data-confidence penalty. In simplified notation reflecting the source, with available dimensions D, weight w_d, raw score s_d, dimension confidence c_d and aggregate confidence C, the score is

$$R = \frac{\sum_{d \in D} w_d s_d c_d}{\sum_{d \in D} w_d}, \qquad S = R\,C^2. \tag{1}$$

The source calculates C with its own weighted-confidence routine and clamps the final score to $[0, 1]$. The symbol S is a ranking score. It is not calibrated to mean, for example, an 80% probability of a comfortable fit. That distinction matters because the UI also uses the word "confidence."

6.2 Ranking, alternatives and explanations

`SizeRecommendationEngine` sorts all valid entries by descending S and picks the first. It then considers alternatives whose score gap is below the greater of a dynamic tolerance and 0.2, taking at most two under the ordinary branch. If weighted confidence is below 0.8, it widens the gap threshold to 0.3 and may show up to three alternatives. Another function analyzes the case in which the top two scores are close. An explanation engine reports dimension deltas and qualitative guidance; a warning path can mention low data confidence or near ties.

This design gives a shopper more than one label when the ranking is close. It is not a guarantee that one of the alternatives will fit. A wrong chart can yield a high-confidence wrong ranking. Conversely, conservative confidence penalties may make every entry score poorly despite a good fit. Both behaviors must be checked against garments actually tried on or otherwise measured. Explanations should remain anchored to the numerical dimensions used in the score, not to retailer marketing language.

7 What the implementation does and does not establish

The repository includes a `BodyDimensionPredictor`, an `MLSizeEngine`, a `SmartSizeParser`, and Android model-management code. Their presence indicates development of further prediction or parsing options, but the active `calculateRecommendation` call uses `BodyEstimator`'s formula path. Likewise, the active image method does not provide the optional network arguments to the OCR engine. We have not relabeled all those components as the path a user currently experiences. This distinction prevents an architectural diagram from overstating deployed capability.

The embedded regional chart is useful as a fallback, but its exact numerical entries require independent provenance and validation. The source comments refer to sizing standards and market research without an accompanying traceable dataset for every constant. ISO 8559-1 describes how anthropometric dimensions are defined and measured [2]; it does not certify a specific built-in chart, brand conversion or regional multiplier. Any future publication claiming standards compliance should map each chart field to the relevant standard and verify licensing of source tables.

The name "offline" also needs a precise boundary. The active local image call uses the bundled ML Kit library, and the sparse-input estimator and scorer are local functions. We did not instrument all runtime requests of the complete application or prove that every model dependency is present before first launch. The Android dependency declaration uses the bundled `com.google.mlkit:text-recognition` artifact, which Google distinguishes from the

dynamically downloaded Play-services variant [3]. A strong offline claim should still be tested on a clean installation with network disabled, for both camera and gallery inputs, plus every fallback path.

8 Threats to validity and failure cases

Three error types must not be collapsed into a single "AI size guide accuracy" number. First, a body-estimation error is a mismatch between predicted and measured circumferences. Second, chart extraction error is a mismatch between a photographed retailer chart and the reconstructed numeric cells. Third, fit error is a mismatch between the selected garment and a wearer's fit outcome. The output of one stage is the input to the next, so downstream fit error can be dominated by a seemingly small unit or alignment mistake upstream.

Several concrete failure cases are visible from source structure. Height and weight do not identify body shape uniquely. A cropped chart may omit unit labels. A size header may be misrecognized as a number. A bare number defaults to cm. A single value expands to a synthetic interval. A fixed raw OCR confidence can make two recognitions with very different legibility enter the same confidence calculation. Regional factors can interact across estimator and scorer. Garment category may be chosen incorrectly. Missing or contradictory dimensions may survive some guards but alter score weights. These are hypotheses for testing, not measured frequencies.

There are also fairness and representativeness concerns. Gender is represented by a limited enum in the estimator; a user's body need not match either formula family. Hard validation bounds can exclude atypical bodies. Regional population assumptions can obscure within-region diversity. A brand-specific chart should take priority over broad regional assumptions when the two disagree, but the present formula route still uses a regional body multiplier. The limitations are engineering questions with consequences for users; addressing them requires measured populations and explicit product charts.

9 Evaluation protocol for a subsequent study

The present paper is a source-level study. A prospective evaluation would assemble a licensed, consented dataset with body measurements, garment type, brand, product-chart photograph, retailer-provided chart, region and an outcome recorded after trying on the garment. The unit of evaluation should be a person–garment pair, with multiple sizes per garment where feasible. Splits must be by product or retailer rather than random images of the same chart: otherwise a test set can leak nearly identical tables from training or rule tuning. Raw user measurements and images should remain on device or under a documented consent and retention policy.

For body estimation, compare predicted and measured chest, waist, hip, shoulder and inseam using median absolute error and error quantiles, reported by garment category and relevant body strata. For the OCR stage, evaluate element text, row/column assignments, units, dimension names, size-label associations and final cell values separately. A chart-level exact-match measure is useful but severe; cell-level precision/recall and numeric absolute error show where reconstruction fails. Photographs should include clean tables and difficult cases such as perspective, blur, partial crops and mixed-language labels.

For recommendation, compare top-one and top-k choices against wearer-reported or independently fitted outcomes, but state the outcome definition: "fits," "preferred fit" and "not

returned" are not identical. Report performance for general charts and product charts separately. Stratify by retailer, garment, region, body measurements and OCR quality. Record abstention or manual-correction rates because a system that refuses an uncertain chart may be safer than one that always returns a label. The confidence values should be calibrated only if an appropriate outcome dataset exists; a reliability diagram could then compare score bins with observed fit frequency.

Useful ablations include removing OCR reconstruction while preserving human-entered charts; replacing estimated body dimensions with measured dimensions; disabling each regional factor independently; replacing the built-in chart with product-specific tables; and varying garment-physics tolerances. A deterministic regression test should verify that two identical measured bodies and the same product chart do not change unexpectedly when only an unrelated locale setting changes. The test matrix must include clean offline installation, airplane mode, repeated image processing and lifecycle interruption. These experiments are specified so future work can refute the design's assumptions rather than merely demonstrate that it runs.

10 Reproducibility and release boundary

The companion repository publishes the manuscript source and PDF with a focused snapshot of existing Kotlin files and a checksum manifest. A reader can inspect the recommendation method in `SizeViewModel`, follow its calls into `BodyEstimator`, `GeneralSizeChartProvider` and `SizeRecommendationEngine`, and compare the scanned-chart route through `OcrSizeIntelligenceEngine`. The source is unchanged from the author's working tree at snapshot time. The snapshot is not presented as a new Android application; its omitted dependencies and build configuration are documented. Adding fresh stub classes would misrepresent the historical implementation.

To independently reproduce a future experimental result, the repository would additionally need a versioned test corpus, product-chart licensing information, evaluation scripts, device and Android version, bundled model version, network-state protocol and exact commit identifier. Those materials are not included here and no numerical result is asserted. The source manifest permits byte-level comparison of the selected extract. Code availability is distinct from validation and from a claim that the whole commercial application is reproducible.

11 Planned extension: product-link ingestion

The project owner has considered a workflow in which a user pastes a product link and the app extracts the listing's size chart, reconstructs it as structured JSON and optionally uses a small remote model when deterministic extraction cannot resolve the table. This would reduce the need to photograph a chart. It is a proposal, not functionality demonstrated by the copied Size Guide route. It would also cross the offline boundary and create new problems: retailer terms and robots restrictions, page changes, anti-bot behavior, localization, ambiguity between model and garment dimensions, remote service cost and privacy of the submitted URL. Any implementation should separate local parsing from an explicitly consented remote fallback and retain the ability to correct cells before recommendation.

The clearest near-term research path is less ambitious: collect licensed chart images with known cells, quantify the current OCR and table error, and test whether measured-body

overrides improve fit selection. That work would establish where a learned component adds value. It should precede claims that a cloud model, more elaborate scraping, or a wider recommendation engine improves decisions.

12 Conclusion

This Android size guide combines sparse user inputs, optional direct body measurements, a general-chart fallback, local text recognition of photographed charts, explicit validation and measurement-level fit ranking. Its architectural strength is that the path from input to selected label is inspectable. Its principal scientific limitation is the lack of reported outcome validation for body formulae, chart reconstruction, regional factors and garment fit. The active estimator's bypass of a trained predictor and the contradictory regional-body comments are important facts, not details to omit. Publishing the original focused source alongside this paper enables independent scrutiny and sets a concrete baseline for the controlled evaluations that should follow.

A A worked source trace without an accuracy claim

Consider a hypothetical user who selects a T-shirt, male formula branch and EU region, enters a height of 170 cm and weight of 65 kg, and does not provide a body circumference or product chart. The example is an arithmetic trace of source constants, not a fit recommendation for a real person. The BMI guard computes $65/(1.70)^2 \simeq 22.49$, inside the software's accepted 10–60 interval. The selected garment type makes chest and waist relevant to the body-estimation branch; hip is not used for this upper-body case. With EU body factors equal to one, the coded formulae produce

$$\widehat{\text{chest}} = 0.52(170) + 0.32(65) - 15 = 94.2 \text{ cm}, \tag{2}$$

$$\widehat{\text{waist}} = 0.45(170) + 0.28(65) - 12 = 82.7 \text{ cm}. \tag{3}$$

These numbers describe the calculation only. They have not been compared with this hypothetical person's actual circumference; the values may be wrong for that person despite passing input validation. If the person supplies a valid chest measurement of 100 cm, the estimator selects the supplied value for chest, leaves waist on its formula path, and records different provenance for the two dimensions. This is the most direct way to see the value of optional measurements: they selectively replace, rather than simply decorate, the sparse-input estimate.

With no product chart, the ViewModel calls `GeneralSizeChartProvider`. In its EU male top table, the entry labelled M lists chest 90–98 cm and waist 76–84 cm; the entry labelled L lists chest 98–106 cm and waist 84–92 cm. It would be tempting to announce M from the two formula outputs. The program does more: it adjusts intervals for garment rules, uses type and region tolerances, scores applicable dimensions, discounts uncertain data, ranks all entries and may return alternatives. A paper reporting the final label must execute this complete pipeline at an identified commit and device environment. This trace stops before that label to avoid silently replacing the implemented scorer with a hand comparison.

The same body input against a photographed product chart takes a different data route. Suppose a visible "M" row is followed by chest "96–100 cm" and waist "80–84 cm." ML Kit

first returns text elements and bounding boxes. The chart engine then has to assign both numbers to the M row and to their respective columns. A token transcription can be correct while a row assignment is wrong. If the unit label is missing, the parser's default of centimeters may appear to work, but an inch-denominated chart would be catastrophically misinterpreted. The example thus separates digit recognition, cell association, unit inference and fit ranking as four distinct checks.

B Audit questions for a reproducible follow-up

The selected source allows a reviewer to formulate falsifiable checks before collecting a dataset. The checks below are not claims that the current system passes. They are a practical bridge from source reading to a scientific protocol.

1. **Input provenance.** Given a valid direct waist circumference, does the estimator always use it instead of the formula result? Does the output preserve the `MEASURED` method marker? Test both upper and lower garments, since an inapplicable dimension may be stored as zero.
2. **Regional consistency.** Hold height, weight, gender and a product chart fixed while changing only region. Compare the raw body estimate, transformed garment interval, tolerance and final ranking separately. This identifies whether regional assumptions are applied once or multiple times.
3. **Chart precedence.** With the same body input, compare the embedded general chart route with a manually supplied product chart. Confirm that the product chart reaches the scorer unchanged by the general-chart fallback, and that labels are returned from the chart actually scored.
4. **OCR cell association.** Use photographs with a ground-truth table transcription. Perturb row spacing, column spacing, rotation and headings one at a time. Inspect reconstructed cell coordinates and values before running the size recommender; a final label alone cannot localize the parsing failure.
5. **Unit handling.** For the same numerical intervals expressed in cm and inches, compare scorer inputs after conversion. Include single values with a unit, ranges with a unit and bare numbers. A bare number should be reported as an assumption, not verified chart metadata.
6. **Uncertainty meaning.** If OCR token quality worsens, determine whether the internal score falls monotonically. Only a separate outcome set can test whether the reported confidence is calibrated to correct fit or correct chart extraction.
7. **Offline behavior.** Install on a clean device with no preloaded dynamic models; disable network access before launching the Size Guide. Repeat for first use, image parsing and a failure/retry case, while monitoring outbound connections. The repository extract alone cannot prove this property.

One should also record negative cases rather than discarding them: charts with no recognized size, conflicting unit marks, mixed children's and adults' rows, measurements of garment dimensions rather than body dimensions, and duplicated product variants. Failure to return a recommendation in these cases may be appropriate. The benchmark should therefore track coverage and abstention alongside accuracy. This avoids improving an average score merely by hiding the inputs on which the system is uncertain.

C Source inventory and interpretation rules

The repository's `source/` directory preserves the original relative paths. It contains the presentation files that drive the Size Guide screen, the models used to represent body and chart entries, the estimator and recommendation modules, OCR extraction and parsing modules, selected UI helpers, and existing tests. The checksum file permits a reader to identify a changed file even if a later revision of the paper edits only prose. No Kotlin source in the snapshot was authored for this paper. Import references that resolve only in the full application remain as historical facts; they are not silently replaced by mock services.

The presence of source files should be read with a call-graph test. The recommendation method in `SizeViewModel` invokes the active estimator and recommender; the ViewModel's local OCR method invokes the OCR engine without optional remote arguments. Other source files may be legacy, experimental or invoked by different screens. A reader should not infer use of a predictive model merely because an `MLSizeEngine` class exists, nor infer that remote parsing occurs merely because an API-service parameter is accepted. Conversely, the source extract's omission of application wiring is a limit of the release, not evidence that the full Android app lacks that wiring.

Finally, the selected existing tests have different evidentiary roles. A unit test can catch a regression in a known formula or a comparator. An instrumented test can demonstrate that a parser runs on a specified Android device. Neither proves that a particular retailer's L fits a person who wears M elsewhere. Such a claim would need a prospective, consented product-and-body outcome study with predeclared metrics. The research snapshot is a foundation for that study, and its most valuable property is that its assumptions can be inspected and challenged.

References

[1] R. Guigourès, Y. K. Ho, E. Koriagin, A.-S. Sheikh, U. Bergmann, and R. Shirvany, "A Hierarchical Bayesian Model for Size Recommendation in Fashion," *Proceedings of the 12th ACM Conference on Recommender Systems*, 2018, pp. 392–396. doi:10.1145/3240323.3240388.

[2] International Organization for Standardization, *ISO 8559-1:2017, Size designation of clothes—Part 1: Anthropometric definitions for body measurement*, 2017. https://www.iso.org/standard/61686.html.

[3] Google, "Recognize text in images with ML Kit on Android," developer documentation. https://developers.google.com/ml-kit/vision/text-recognition/v2/android.

[4] B. Smock, R. Pesala, and R. Abraham, "PubTables-1M: Towards Comprehensive Table Extraction From Unstructured Documents," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 4634–4642. doi:10.1109/CVPR52688.2022.00459.